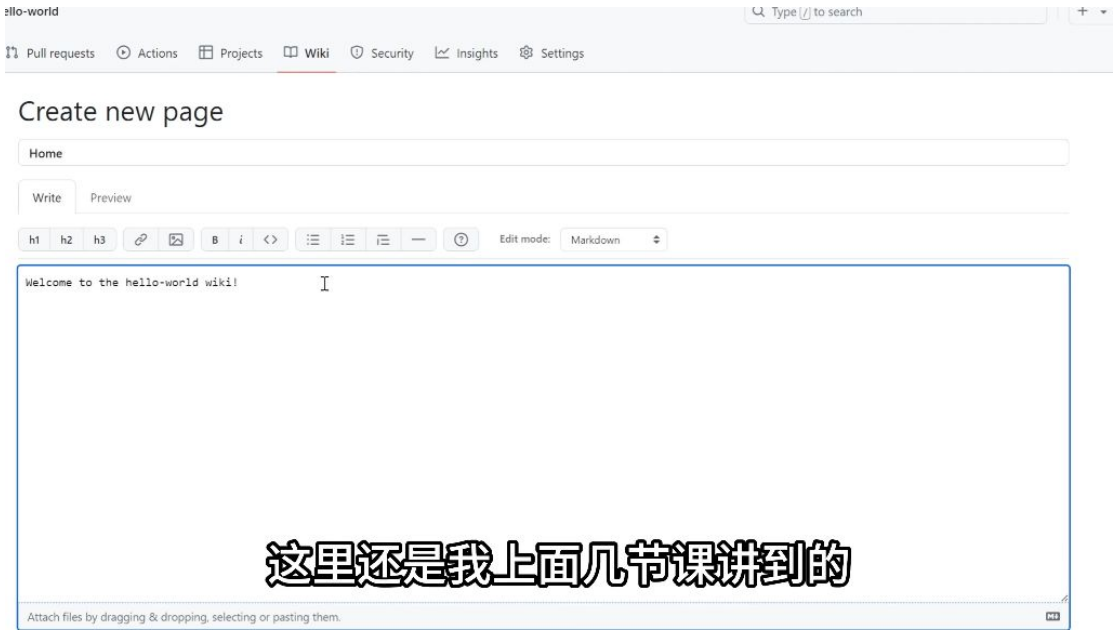
-- 以下为AI生成的图文笔记内容 --

*一、仓库的其他功能**00:00***

1. Github Wiki 00:11

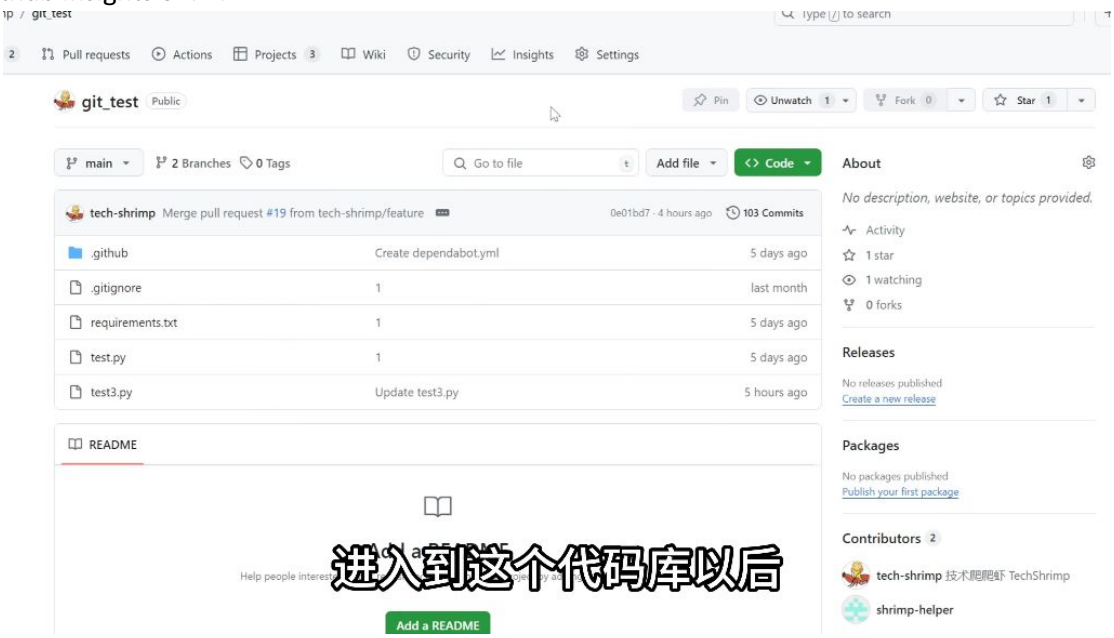
- **用途**：用于存放项目的详细文档、指南、API文档、设计文档等结构化内容
- **特点**：
  - 独立Git仓库，可独立克隆和推送
  - 支持Markdown语法编辑
  - 内置导航栏方便浏览搜索文档内容

1）Github Wiki的创建 00:25



- **创建步骤**：
  - 点击仓库头部"Wiki"标签栏进入
  - 通过"New page"创建新页面
  - 使用Markdown语法编辑内容
  - 支持多级标题自动生成导航
- **管理功能**：
  - 右下角显示克隆链接（如：https:/github.com/tech-shrimp/hello）
  - 可添加自定义侧边栏和页脚

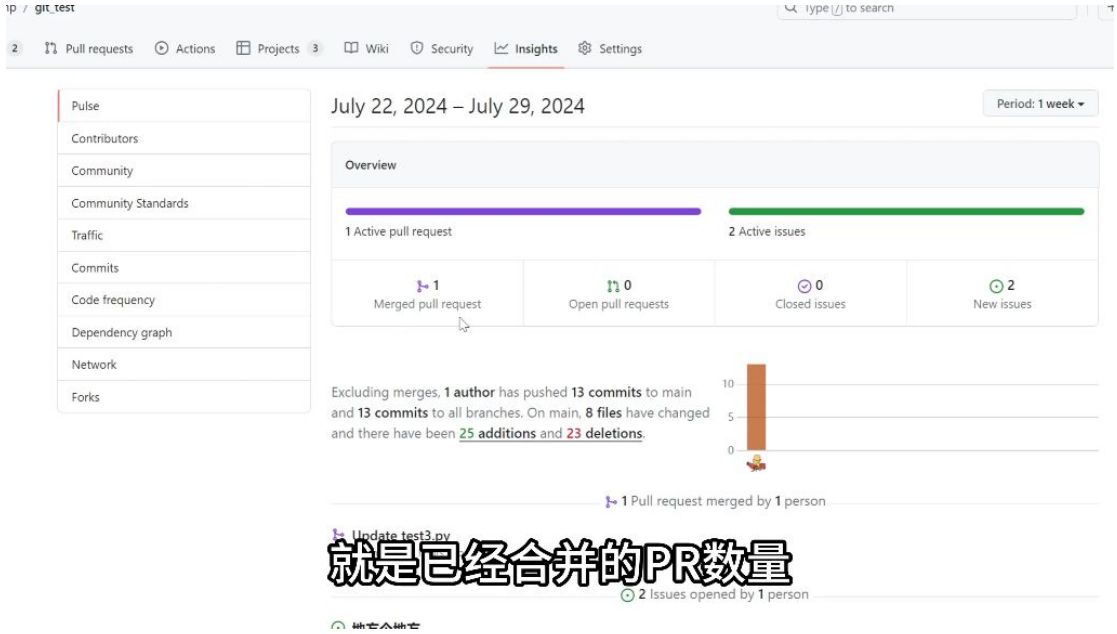
2. Github Insights 01:14



- **核心价值**：帮助项目维护者和贡献者分析代码库活动状态
- **主要模块**：
  - Pulse：项目活动概要
  - Contributors：贡献者统计
  - Traffic：访问量数据
  - Code Frequency：代码变更频率

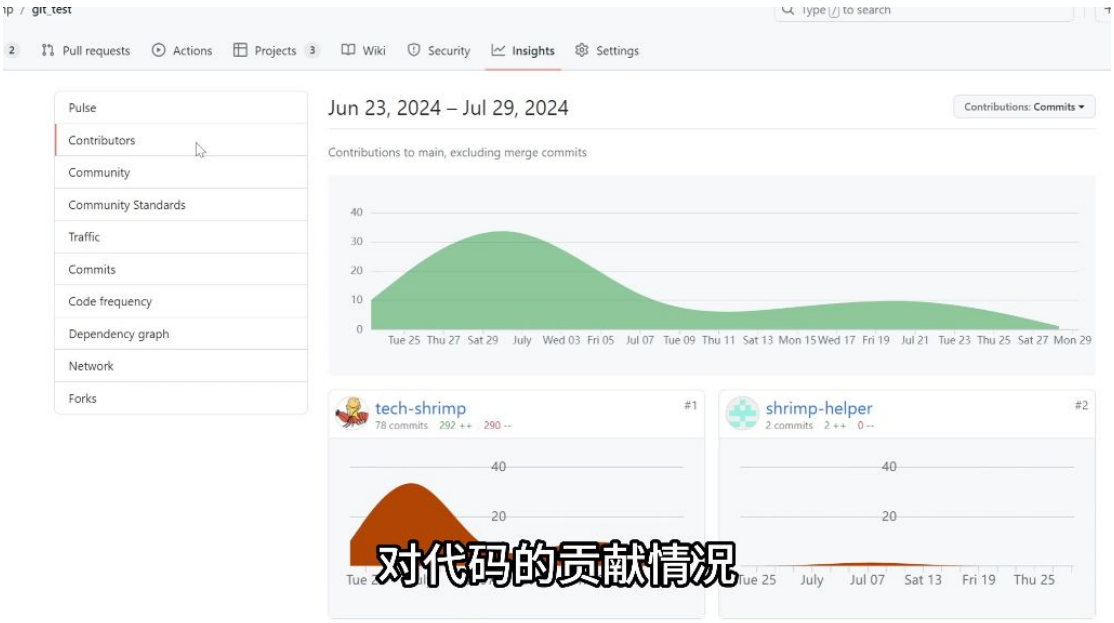
- Dependency Graph：依赖关系图

1）Github Insights的Pulse 01:32

- 

- **显示内容**：
  - 已合并的拉取请求（如示例中显示14个）
  - 开启的拉取请求（如示例中显示2个）
  - 关闭的问题（Closed issues）
  - 新开启的问题（New issues）
- **时间筛选**：支持24小时/3天/1周/1个月等时间范围选择

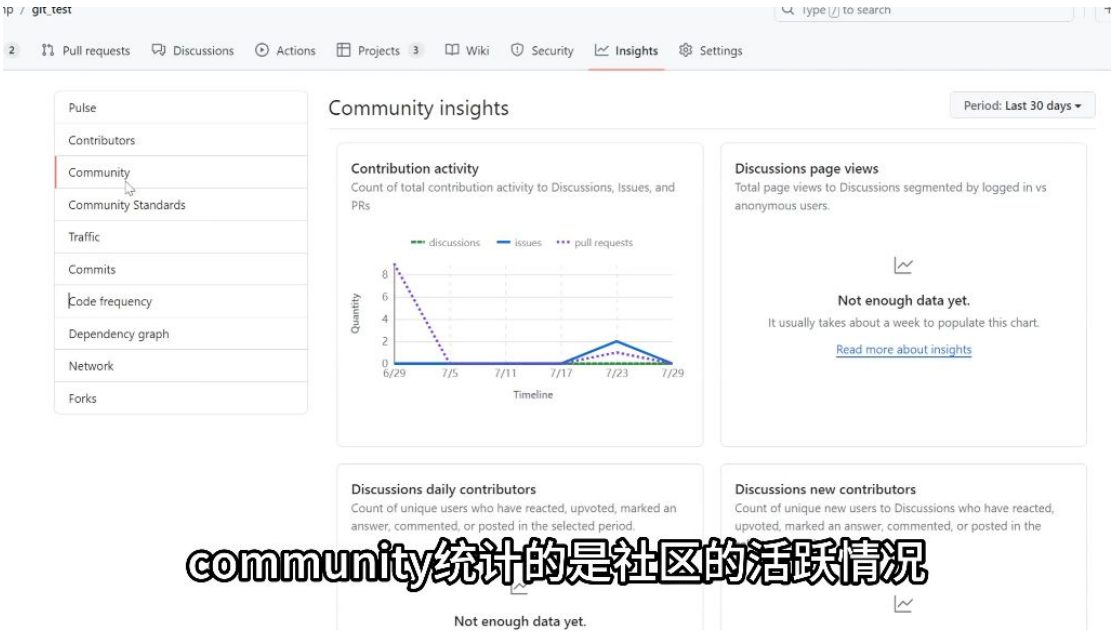
2）Github Insights的Contributors 02:08



-

- **数据维度**：
  - 横轴：时间线（可显示具体日期如Jul 07, Fri 19等）
  - 纵轴：提交次数
  - 显示每位贡献者的提交数、代码行数变化
- **示例数据**：某项目显示2位作者进行了70次对main分支的直接提交

3）Github Insights的Community 02:25



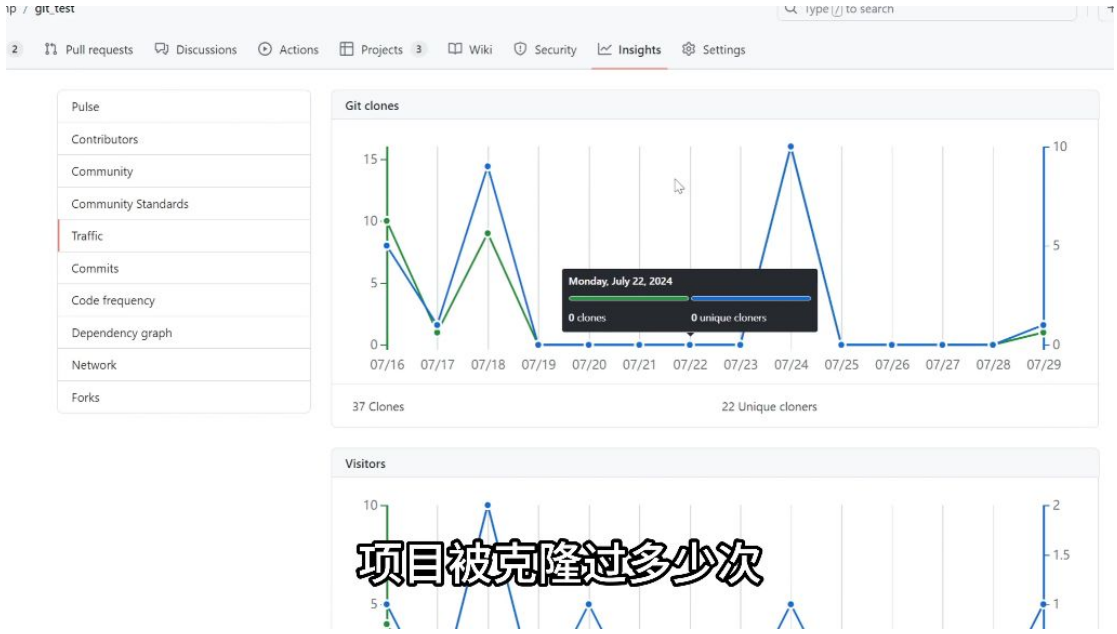
-

- **统计指标**：
  - Discussions页面浏览量

- 每日讨论参与者数量
- 新讨论参与者数量

- **注意事项**：通常需要约1周时间数据才会完整显示

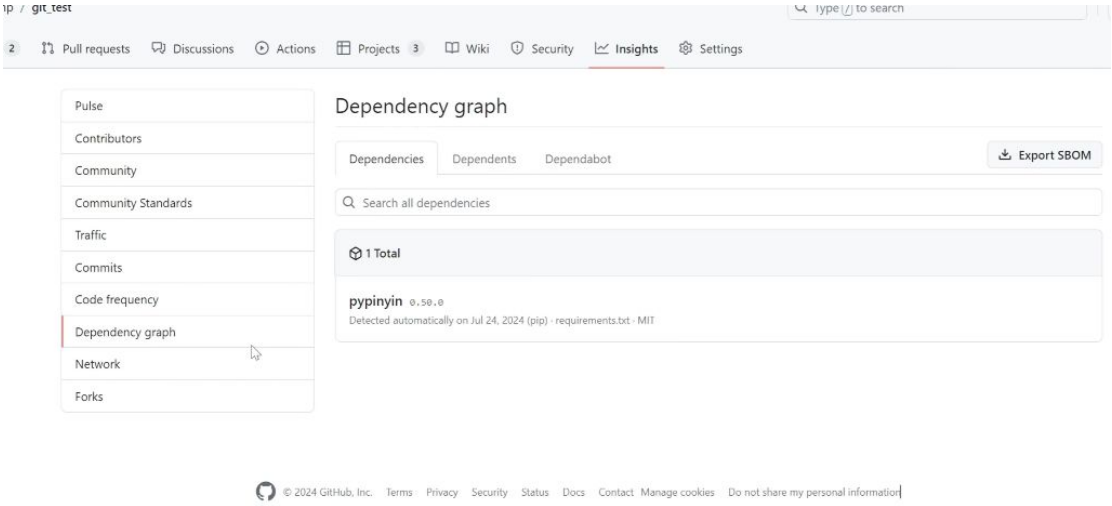
4）Github Insights的Traffic 03:57

- 

- **关键指标**：
  - Git克隆次数（如示例显示37次克隆）
  - 唯一克隆者数量（如示例显示22个）
  - 访客数量

- **数据展示**：支持按日/周查看趋势变化

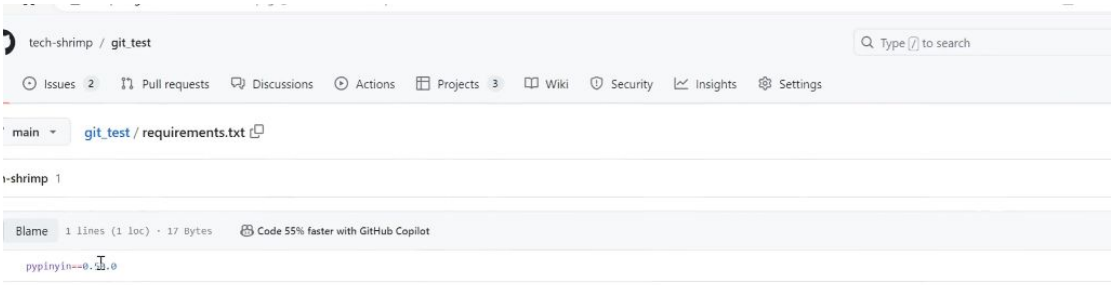
5）Github Insights的Code Frequency 04:12



这里的dependency graph

-
- **可视化方式**：
  - 横轴：时间线（如显示23 July 2024等具体日期）
  - 纵轴：代码行数变化
  - 绿色柱：新增代码行数
  - 红色柱：删除代码行数

6）Github Insights的Dependency Graph 04:21

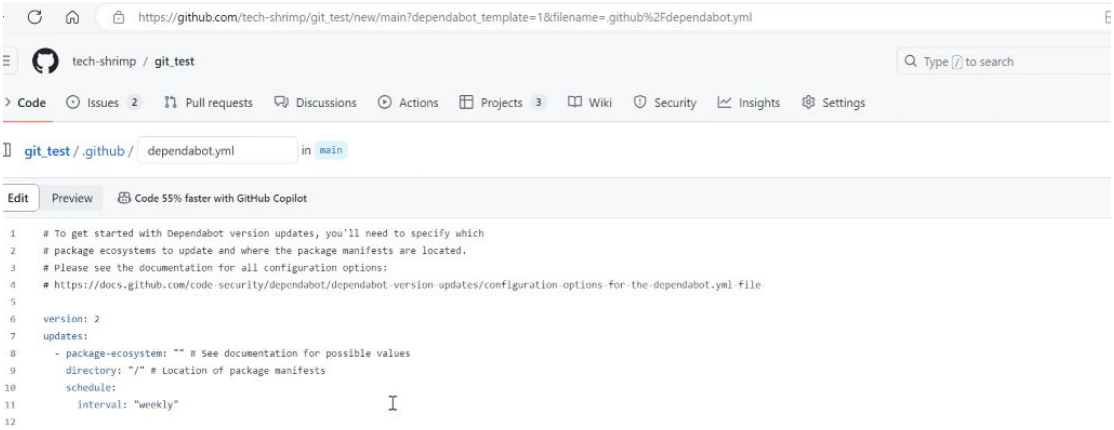


在我这个项目的requirements.txt文件

-
- **三大功能**：
  - Dependencies：显示项目依赖的第三方库（如示例中的pypinyin 0.50.0）

- Dependents：显示项目被其他仓库依赖的情况（如示例中拼音库被6191个仓库依赖）
- Dependabot：自动依赖更新功能

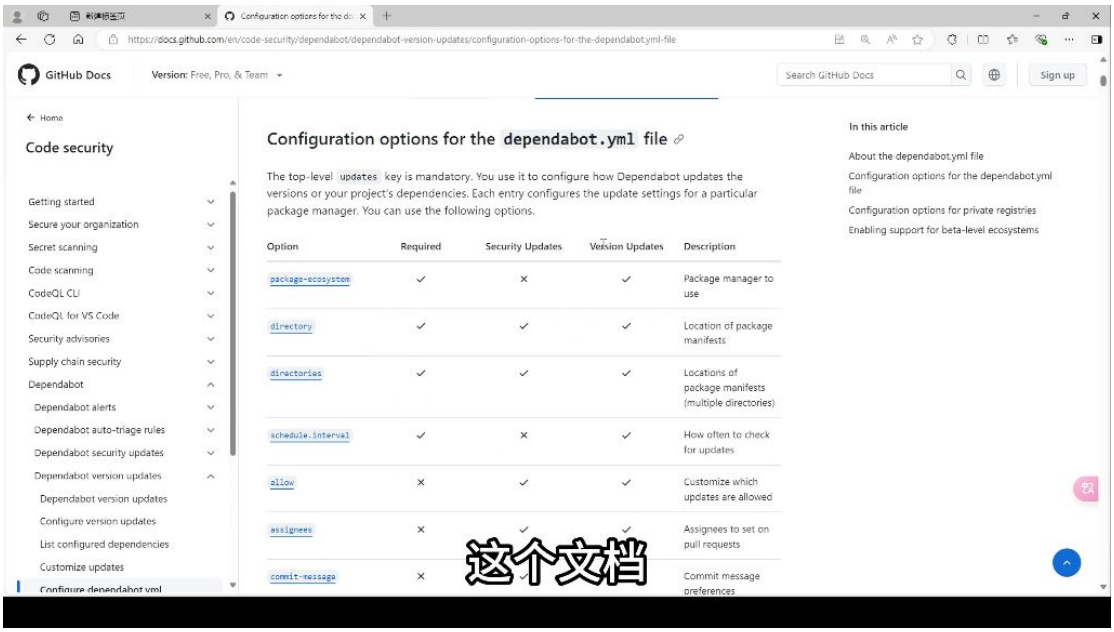
7）Github Insights的其他功能 05:16



它自动为我们创建了一个文件

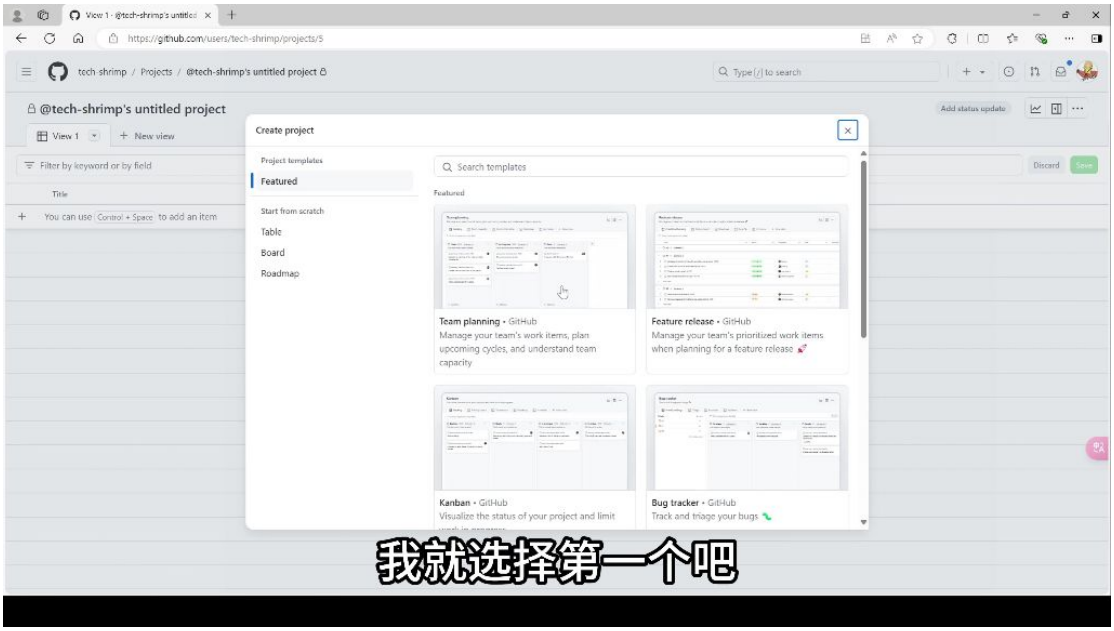
-
- **自动化更新**：
  - 创建.github/dependabot.yml配置文件
  - 主要配置项：
    - package-ecosystem：包管理系统（如pip/npm/maven等）
    - directory：依赖文件目录（通常为"/"）
    - schedule.interval：扫描间隔（weekly/daily/monthly）
- **工作流程**：检测到更新后自动创建Pull Request（如将pypinyin从0.50.0更新到0.51.0）

3. Github Docs 06:56

- 

- **配置参考**：
  - 官方文档地址：https:/docs.github.com/en/code-security/dependabot
  - 高级功能：
    - groups：对更新规则分组（如区分版本更新和安全更新）
    - ignore：忽略特定版本更新（如忽略major版本更新）

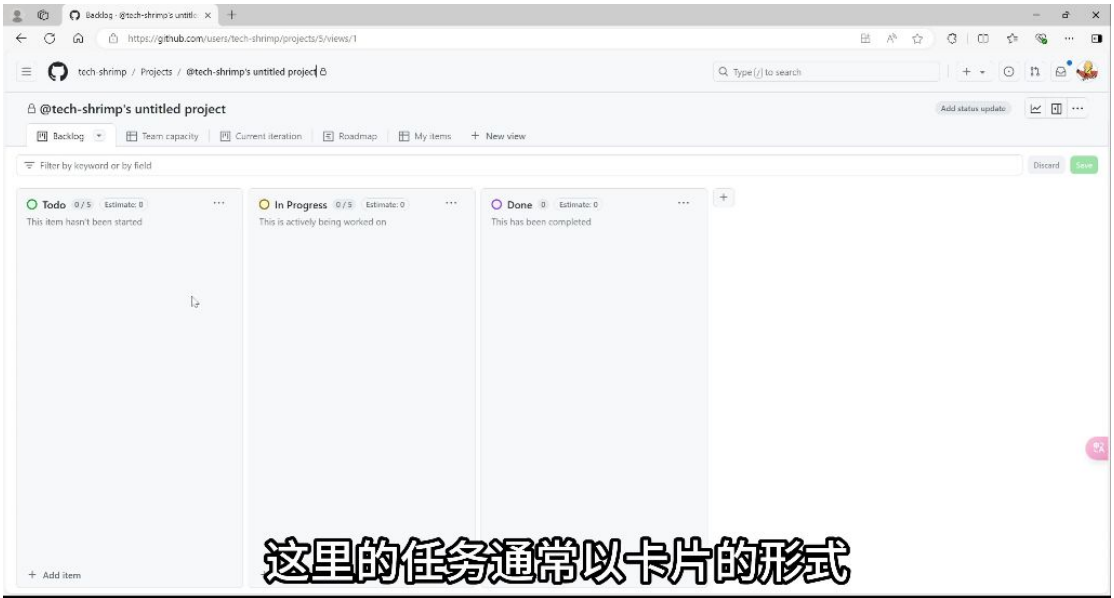
4. GitHub Project 09:09

- 

- **核心功能**：可视化管理工具，用于组织和跟踪项目进度
- **主要视图**：

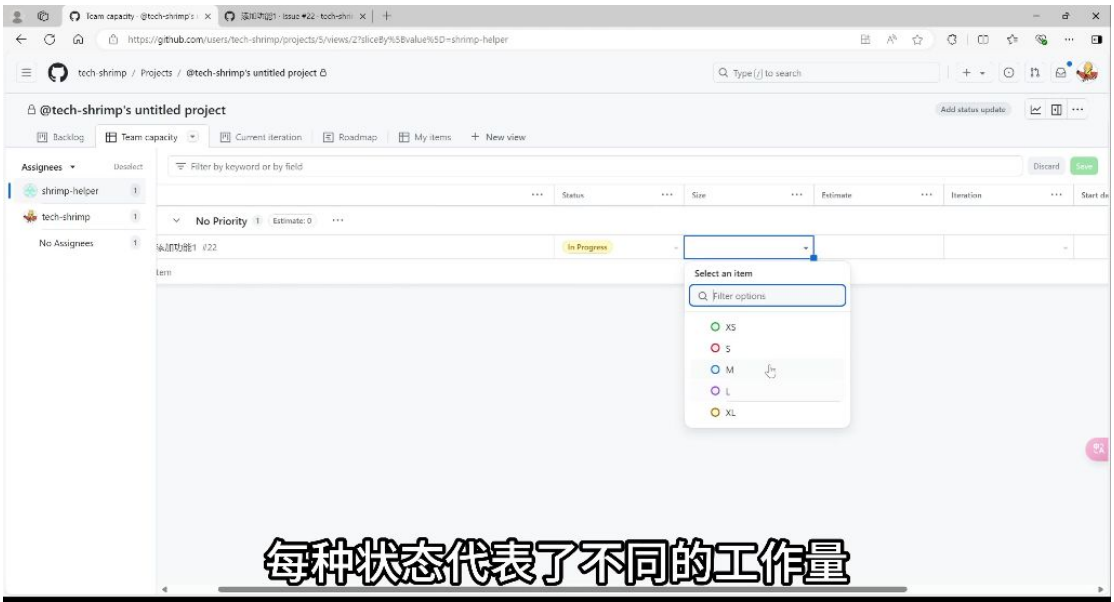
- Table：表格视图
- Board：看板视图
- Roadmap：路线图视图

1）GitHub Project的创建 09:23

- 

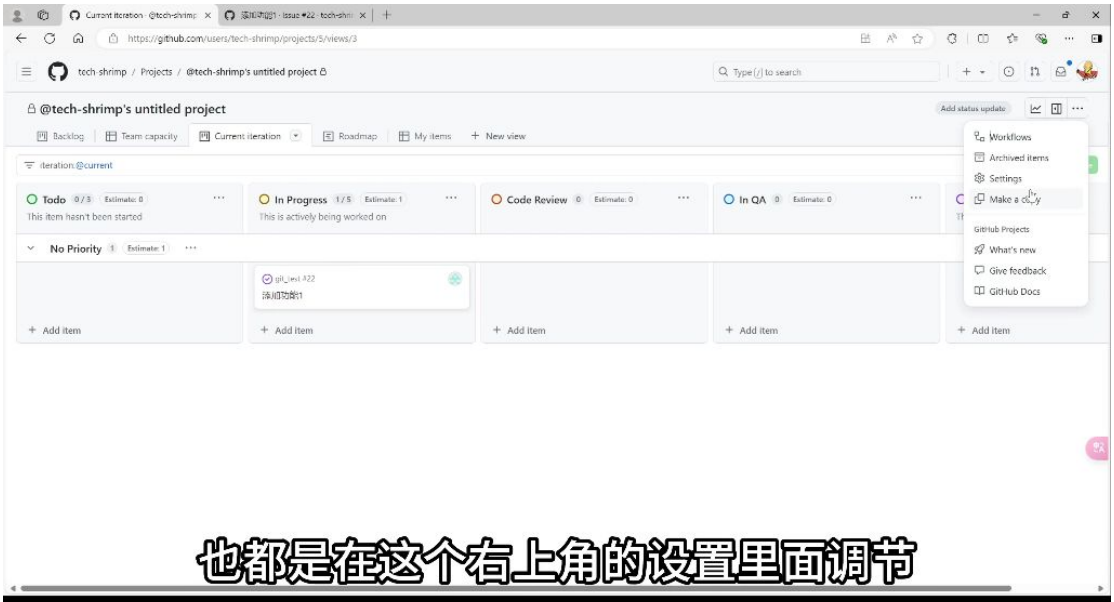
- **创建步骤**：
  - 点击仓库"Projects"标签
  - 选择模板（如Kanban/Team planning等）
  - 设置初始列（如To Do/In Progress/Done）
- **任务管理**：
  - 支持拖拽方式变更任务状态
  - 可添加自定义状态列（如Code Review/In QA等）

2）GitHub Project的Team Capacity 12:04

- 

- **管理维度**：
  - 优先级（P0-P2等级）
  - 任务规模（XS-XXXL）
  - 预估时间（如1天/3天等）
  - 迭代周期（Sprint设置）
- **分配功能**：可将任务分配给特定成员（如示例中的shrimp-helper）

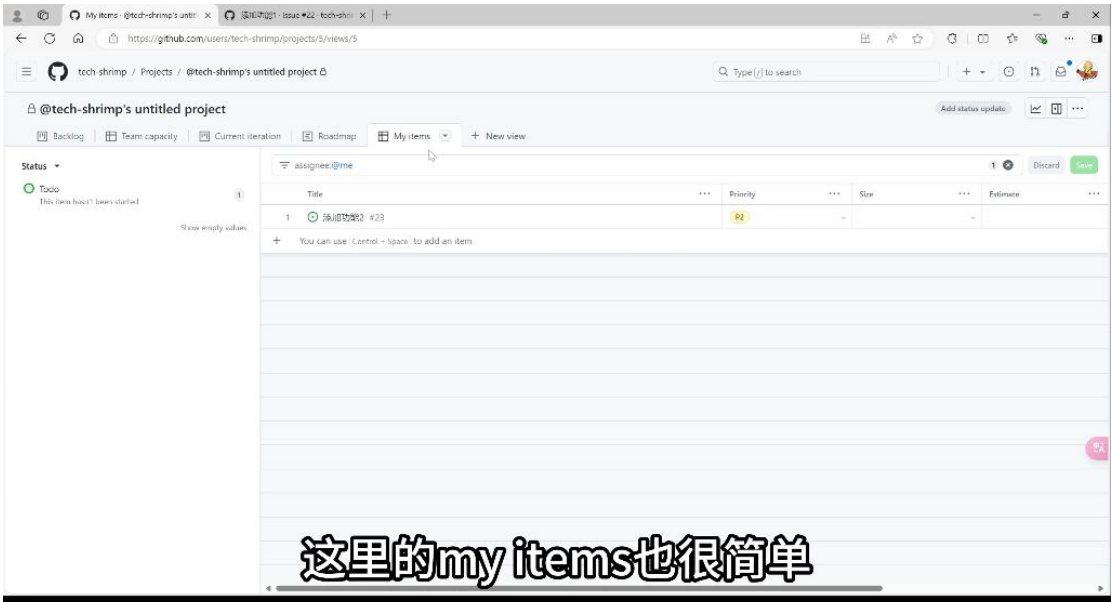
3）GitHub Project的Current Iteration 14:07

- 

- **敏捷开发**：

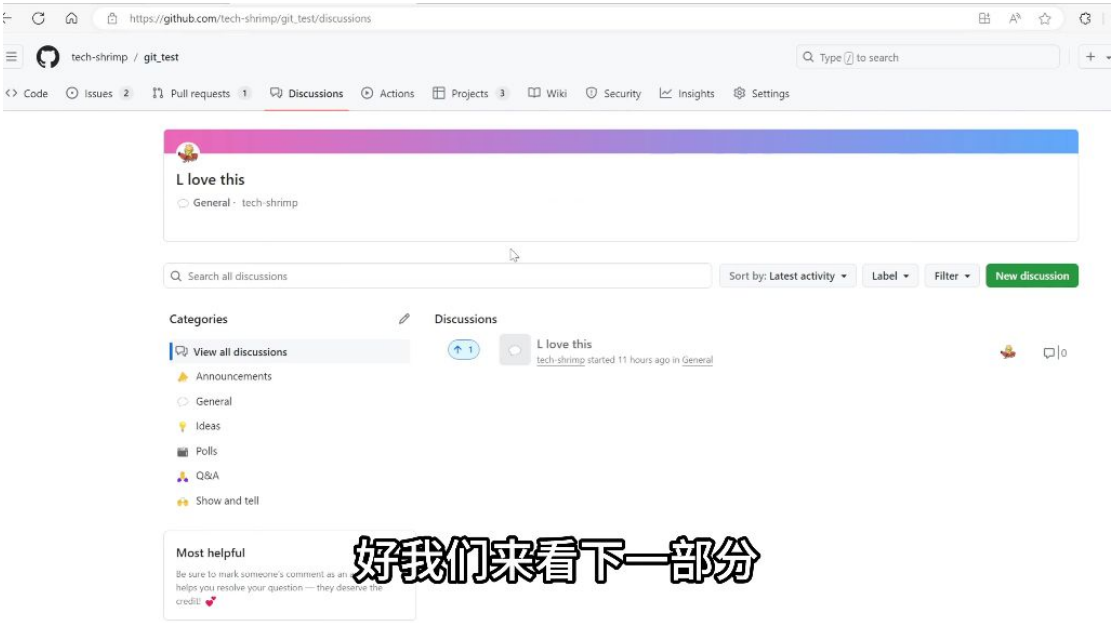
- 典型迭代周期：2周
- 显示当前迭代所有任务状态
- 支持任务状态流转（To Do → In Progress → Done）

4）GitHub Project的Roadmap 14:28

- 

- **时间规划**：
  - 按月/季度显示任务排期
  - 支持设置开始/结束日期
  - 可视化任务依赖关系

5）GitHub Project的My Items 14:37



-
- **个人工作台**：
  - 筛选assignee:@me查看分配任务
  - 支持快速添加新任务（Control+Space快捷键）
  - 显示任务优先级和状态概览

二、知识小结

知识点	**核心内容**	**功能亮点**	**适用场景**
GitHub维基	- 项目的详细文档存储空间 - 支持Markdown语法 - 独立Git仓库可克隆推送	- 结构化文档管理 - 内置导航栏和搜索功能 - **实时协作编辑**	大型项目文档维护/API文档托管
GitHub Insights	- 代码库活动数据分析 - 包含合并PR数、提交频率等指标 - 贡献者活跃度可视化	- **多维度统计图表** - 时间范围筛选（日/周/月） - 社区标准检查清单	项目维护者监控/开源社区健康度评估
Dependency Graph	- 依赖项版本管理 - 自动扫描更新（Dependabot） - 安全补丁与版本更新分离配置	- **自动创建PR更新依赖** - 支持Python/npm等包管理工具 - 忽略特定版本规则（ignore配置）	多依赖项目/安全敏感型代码库

GitHub Projects	- 看板式任务管理 - 支持敏捷开发（迭代/Sprint） - 自定义状态列（To Do/In Progress/Done等）	- **拖拽式任务流转** - 工作量估算（Size/Estimate） - 优先级标记（P0-P2）	团队协作/敏捷开发项目管理
GitHub Discussions	- 项目论坛功能 - 发帖/回帖/分类筛选 - 标签与搜索支持	- **社区化交流平台** - 与Issues功能互补 - 主题分类（Q&A/Ideas等）	大型开源项目社区讨论/功能提案收集